

Smartphone IMU-Based Real-Time Posture Correction

Mobile Computing and Its Application
Mini Project



Seoul National University
Spring 2026

Presented by
HyunJong Park



Detect poor neck posture during smartphone use



Usage Scenario



Standing



Sitting



Commuting



Problem

Users often maintain poor neck posture when using smartphones, which puts strain on the neck and can lead to neck pain and forward head posture.



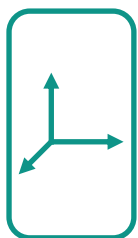
Goal

Develop a lightweight, real-time posture correction system using only built-in smartphone IMU sensors (accelerometer & gyroscope) as an everyday posture-awareness assistant.

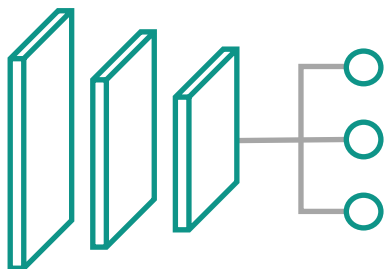
Key Idea

Robust Posture Classification with Temporal Filtering

Sense



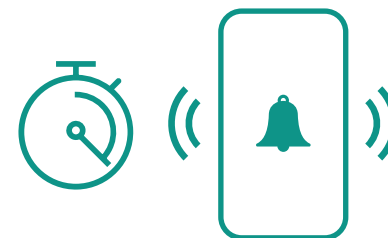
Train



Infer

	t1	t2	t3	avg
●	0.60	0.40	0.55	0.52
●	0.30	0.50	0.10	0.38
●	0.10	0.10	0.10	0.10

Feedback



6-axis IMU input

50 Hz sampling

Ring-buffered
sensor stream

Input: 100 × 6 IMU
window
(2 sec @ 50 Hz)

Lightweight 1D-CNN

Classes: Upright,
Mild Forward Flexion
Severe Looking-Down

Smooth prediction
probabilities

3-result moving average

Reduces transient
misclassification

8-sec sustained
poor-posture check

Vibration +
notification alert

Avoids immediate
false alarms

Posture Correction App – Screen Explanation



1. Model selection

- Before starting monitoring, you can choose between INT8(default) or FP32(fallback)

2. Posture classification

- Shows prediction for current posture (Upright, Mild Forward Flexion, and Severe Looking Down.)
- Shows confidence for each state.

3. Countdown Bar

- Shows countdown progress for persistent bad posture.

4. Monitoring status

- Shows monitoring status: Stopped or Active

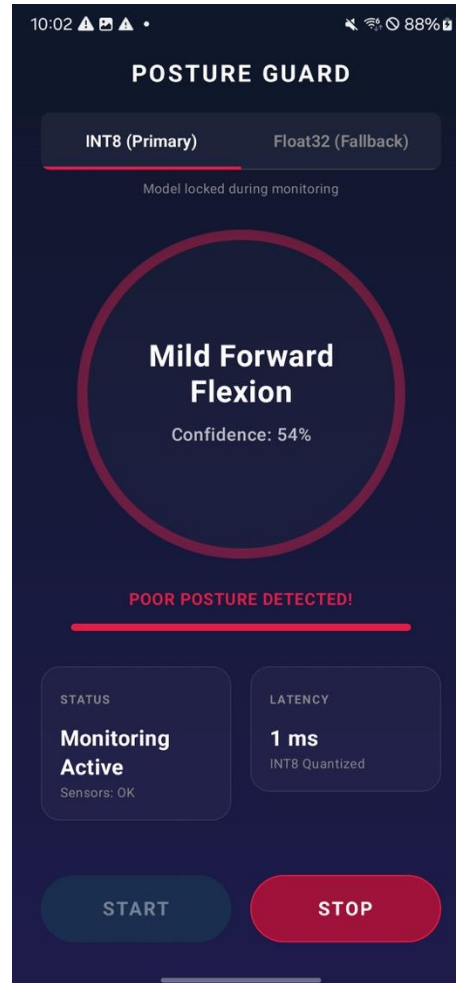
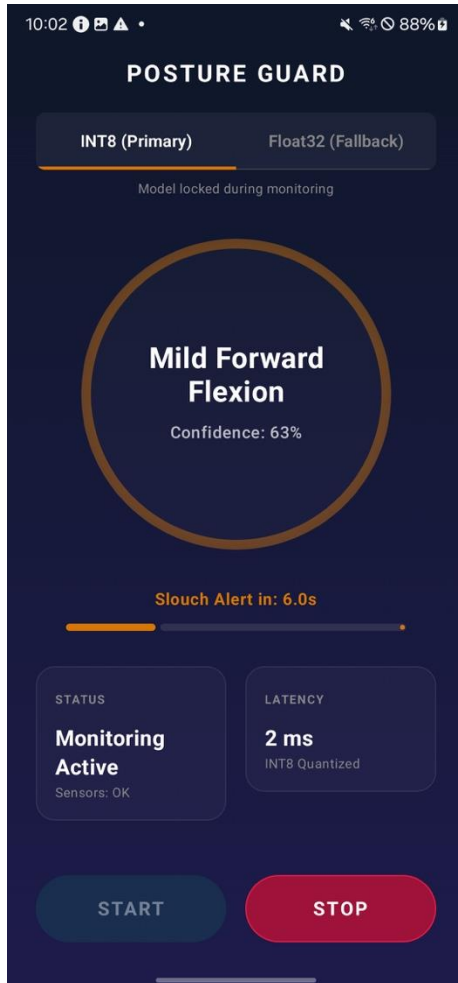
5. Latency

- Shows inference latency of a model

6. Start/Stop monitoring

- Button to start or stop monitoring

Posture Correction App – Practical Features



Alert Rule

If bad posture continues for 8 seconds, the app sends a notification and vibration alert

Foreground Service

Runs a background service to track posture even when using other apps.

Adaptive Vibration Feedback

Vibration intensity and frequency vary depending on posture severity

Screen-State Lifecycle Awareness

Automatically pauses sensor sampling when the screen turns off and resumes on screen unlock to save battery.

Improving Accuracy and Latency

Noisy Boundary Filtering

Metric	Before Filtering	After Filtering
Test Accuracy	90.72%	92.90%
Macro F1	90.68%	92.88%

- *Before (Raw Windows):* 90.72% Accuracy, 90.68% Macro-F1. Slouching detection suffered from transitional noise.
- *After (0.25s / 12-row margin removed):* **92.90% Accuracy (+2.18%)** and **92.88% Macro-F1 (+2.20%)**.

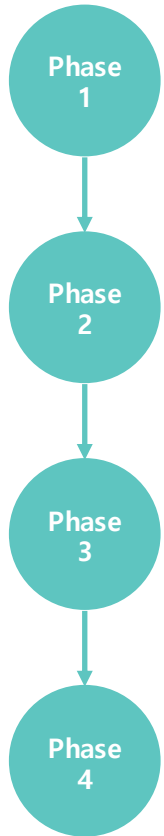
Model Quantization

Model	FP32 TFLite	INT8 TFLite
Size	190 KB	65KB
Latency	0.1485ms	0.1045ms

- **Model Size:** Achieved a **~66% size reduction** for resource-constrained environments.
- **Inference Latency:** On invoke-only latency, the quantized INT8 model was 29.6% faster than the FP32 model on a Samsung Galaxy S23 Ultra.

Full pipeline: collection to real-time inference

Handled the entire pipeline independently, from data collection to app implementation



Tooling & Data Collection

- Developed IMUDataCollectorApp for Android
- Collected 16 separate labeled CSV sessions (posture_session_*.csv) under various contexts

Preprocessing & Training

- Implemented preprocessing pipeline (50 Hz resampling, segmentation, transition margin trimming).
- Performed cross-session test split validation

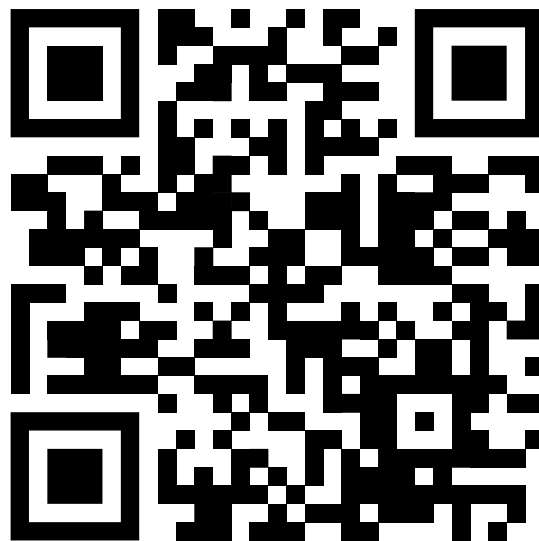
On-Device Integration

- Developed the real-time posture monitoring App
- Integrated TFlite inference

Polish & Wrap-Up

- Polished minor features for practicality
- Prepared final demonstration materials.

Thank You



<https://github.com/13oris/PostureDetection>